Guia Passo a Passo — Classificação e Localização de Danos em Veículos (Deep Learning + CV)

Objetivo: construir, do zero, um modelo de classificação de danos em veículos a partir de fotos de sinistros, com pré-processamento em OpenCV, transfer learning com CNN (ResNet/EfficientNet via PyTorch), avaliação com matriz de confusão, explicabilidade visual com Grad-CAM, publicação no Hugging Face Hub e disponibilização via API com FastAPI.

Dataset: Car Damage Detection (Kaggle) — imagens divididas em duas classes: `00-damage` (com dano) e `01-whole` (sem dano), já separadas em treino e validação.

> **Nota importante sobre "localização":** este dataset não possui bounding boxes nem máscaras de segmentação. A localização do dano será feita de forma **visual e explicável via Grad-CAM**, que gera mapas de calor mostrando **onde** na imagem o modelo "olhou" para decidir. Isso é exatamente o que se espera de explicabilidade em um contexto de sinistro (justificar a decisão do modelo para um analista humano). Ao final do guia há uma extensão opcional com segmentação real, caso queira habilitar também esse checkbox.

Etapa 0 — Estrutura do projeto e ambiente

0.1. Crie a estrutura de pastas

```
car-damage-detection/
├── data/                   # dataset (não versionar no git)
├── notebooks/              # exploração e experimentos
│   └── 01_eda.ipynb
├── src/
│   ├── preprocessing.py    # funções OpenCV
│   ├── dataset.py          # Dataset/DataLoader PyTorch
│   ├── train.py            # loop de treino
│   ├── evaluate.py         # métricas e matriz de confusão
│   └── gradcam.py          # explicabilidade
├── app/
│   └── main.py             # API FastAPI
├── models/                 # checkpoints (.pt)
├── outputs/                # gráficos, heatmaps, relatórios
├── requirements.txt
├── README.md
└── .gitignore
```

0.2. Crie e ative o ambiente virtual

bash

```
python -m venv .venv
source .venv/bin/activate        # Linux/Mac
# .venv\Scripts\activate         # Windows
```

0.3. Instale as dependências

Crie o `requirements.txt`:

```
torch
torchvision
opencv-python
albumentations
numpy
pandas
matplotlib
seaborn
scikit-learn
grad-cam
huggingface_hub
fastapi
uvicorn[standard]
python-multipart
pillow
kaggle
```

bash

```
pip install -r requirements.txt
```

Dica: se tiver GPU NVIDIA, instale o PyTorch com CUDA seguindo o seletor oficial em pytorch.org. O projeto roda em CPU, mas o fine-tuning fica muito mais rápido em GPU (ou use Google Colab/Kaggle Notebooks com GPU gratuita).

Etapa 1 — Download e exploração do dataset

1.1. Configure a API do Kaggle

1. Em kaggle.com → Settings → API → **Create New Token** (baixa o `kaggle.json`).
2. Coloque o arquivo em `~/.kaggle/kaggle.json` e ajuste as permissões:

bash

```
mkdir -p ~/.kaggle
mv ~/Downloads/kaggle.json ~/.kaggle/
chmod 600 ~/.kaggle/kaggle.json
```

1.2. Baixe e extraia o dataset

```bash
kaggle datasets download -d anujms/car-damage-detection -p data/
unzip data/car-damage-detection.zip -d data/
```

A estrutura esperada após extração (pasta `data1a`):

```
data/data1a/
├── training/
|   ├── 00-damage/
|   └── 01-whole/
└── validation/
    ├── 00-damage/
    └── 01-whole/
```

1.3. Análise exploratória (EDA)

No notebook `01_eda.ipynb`, responda:

- **Quantas imagens por classe e por split?** Verifique balanceamento (este dataset é razoavelmente balanceado, ~920 imagens por classe no treino). Se houver desbalanceamento forte, anote — isso influencia a escolha de métricas e de `class_weights` na loss.
- **Quais as dimensões das imagens?** Plote a distribuição de largura × altura. Elas variam bastante, o que justifica o resize padronizado.
- **Como são as imagens?** Visualize uma grade de exemplos de cada classe. Observe ruídos típicos: fundos variados, ângulos diferentes, iluminação inconsistente, reflexos — isso motiva o pré-processamento e o data augmentation.

```python
import os
from collections import Counter

base = "data/data1a"
for split in ["training", "validation"]:
    counts = {c: len(os.listdir(os.path.join(base, split, c)))
              for c in os.listdir(os.path.join(base, split))}
    print(split, counts)
```

Entregável da etapa: notebook com contagens, grade de imagens e 3–5 observações escritas sobre a qualidade dos dados.

Etapa 2 — Pré-processamento com OpenCV

Aqui você constrói funções reutilizáveis em `src/preprocessing.py`. O objetivo é padronizar as imagens e melhorar contraste/nitidez antes do modelo.

2.1. Pipeline sugerido

1. **Leitura e conversão de cor** — OpenCV lê em BGR; converta para RGB.
2. **Resize com preservação de proporção + padding** — evita distorcer o carro (alternativa simples: resize direto para 224×224).
3. **Redução de ruído** — filtro bilateral preserva bordas (importante: amassados e arranhões são detalhes de borda).
4. **Equalização de contraste (CLAHE)** — melhora fotos escuras/estouradas, comuns em fotos de sinistro tiradas com celular.

```python
```

```python
# src/preprocessing.py
import cv2
import numpy as np

IMG_SIZE = 224

def load_image(path: str) -> np.ndarray:
    img = cv2.imread(path)
    if img is None:
        raise ValueError(f"Não foi possível ler: {path}")
    return cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

def resize_with_padding(img: np.ndarray, size: int = IMG_SIZE) -> np.ndarray:
    h, w = img.shape[:2]
    scale = size / max(h, w)
    nh, nw = int(h * scale), int(w * scale)
    img = cv2.resize(img, (nw, nh), interpolation=cv2.INTER_AREA)
    top = (size - nh) // 2
    bottom = size - nh - top
    left = (size - nw) // 2
    right = size - nw - left
    return cv2.copyMakeBorder(img, top, bottom, left, right,
                              cv2.BORDER_CONSTANT, value=(0, 0, 0))

def denoise(img: np.ndarray) -> np.ndarray:
    return cv2.bilateralFilter(img, d=9, sigmaColor=75, sigmaSpace=75)

def apply_clahe(img: np.ndarray) -> np.ndarray:
    lab = cv2.cvtColor(img, cv2.COLOR_RGB2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    l = clahe.apply(l)
    return cv2.cvtColor(cv2.merge([l, a, b]), cv2.COLOR_LAB2RGB)

def preprocess(path: str) -> np.ndarray:
    img = load_image(path)
    img = resize_with_padding(img)
    img = denoise(img)
    img = apply_clahe(img)
    return img
```

2.2. Valide visualmente

No notebook, plote lado a lado **original vs. pré-processada** para ~10 imagens. Confirme que o CLAHE está ajudando (e não criando artefatos). Se estiver exagerado, reduza `clipLimit`.

Ponto de atenção: o pré-processamento deve ser **idêntico no treino, na avaliação e na API**. Centralizar tudo em `preprocess()` garante isso.

Etapa 3 — Data augmentation e Dataset PyTorch

3.1. Defina as transformações com Albumentations

Augmentation só no **treino** — validação usa apenas normalização. Escolha transformações plausíveis para fotos de sinistro (o segurado pode fotografar de qualquer ângulo, com qualquer luz):

```python
# src/dataset.py
import albumentations as A
from albumentations.pytorch import ToTensorV2

MEAN = (0.485, 0.456, 0.406)   # estatísticas do ImageNet
STD  = (0.229, 0.224, 0.225)   # obrigatórias em transfer learning

train_tfms = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.Rotate(limit=15, p=0.5),
    A.RandomBrightnessContrast(brightness_limit=0.2, contrast_limit=0.2, p=0.5),
    A.HueSaturationValue(hue_shift_limit=10, sat_shift_limit=20, p=0.3),
    A.GaussNoise(p=0.2),
    A.CoarseDropout(max_holes=4, max_height=24, max_width=24, p=0.3),
    A.Normalize(mean=MEAN, std=STD),
    ToTensorV2(),
])

val_tfms = A.Compose([
    A.Normalize(mean=MEAN, std=STD),
    ToTensorV2(),
])
```

Por que essas escolhas: flip horizontal e rotação leve simulam ângulos de câmera; brilho/contraste simula iluminação; `CoarseDropout` força o modelo a não depender de uma única região; ruído gaussiano simula fotos de celular de baixa qualidade. Evite flips verticais (carro de cabeça para baixo não é realista) e rotações grandes.

3.2. Implemente o Dataset customizado

```python
```

```python
import os
import torch
from torch.utils.data import Dataset, DataLoader
from src.preprocessing import preprocess

CLASSES = ["00-damage", "01-whole"]  # 0 = dano, 1 = íntegro

class CarDamageDataset(Dataset):
    def __init__(self, root: str, transforms=None):
        self.samples = []
        self.transforms = transforms
        for label, cls in enumerate(CLASSES):
            folder = os.path.join(root, cls)
            for fname in os.listdir(folder):
                self.samples.append((os.path.join(folder, fname), label))

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        path, label = self.samples[idx]
        img = preprocess(path)                    # OpenCV pipeline
        if self.transforms:
            img = self.transforms(image=img)["image"]
        return img, torch.tensor(label, dtype=torch.long)

def get_loaders(batch_size=32, num_workers=2):
    train_ds = CarDamageDataset("data/data1a/training", train_tfms)
    val_ds   = CarDamageDataset("data/data1a/validation", val_tfms)
    train_dl = DataLoader(train_ds, batch_size=batch_size, shuffle=True,
                          num_workers=num_workers, pin_memory=True)
    val_dl   = DataLoader(val_ds, batch_size=batch_size, shuffle=False,
                          num_workers=num_workers, pin_memory=True)
    return train_dl, val_dl
```

3.3. Sanidade

Itere um batch e verifique: shape `[32, 3, 224, 224]`, dtype `float32`, labels em `{0, 1}`. Plote imagens **desnormalizadas** de um batch de treino para conferir se o augmentation está razoável.

Etapa 4 — Transfer learning com ResNet/EfficientNet

4.1. Estratégia

Transfer learning em duas fases:

1. **Fase 1 (feature extraction):** congele todo o backbone pré-treinado no ImageNet e treine apenas a nova camada final. Poucas épocas, learning rate maior (ex.: 1e-3).
2. **Fase 2 (fine-tuning):** descongele as últimas camadas (ou tudo) e treine com learning rate bem menor (ex.: 1e-5 a 1e-4) para ajustar as features ao domínio de carros danificados sem "esquecer" o ImageNet.

4.2. Monte o modelo

python

```python
# src/train.py
import torch
import torch.nn as nn
from torchvision import models

def build_model(arch: str = "resnet50", num_classes: int = 2):
    if arch == "resnet50":
        model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
        for p in model.parameters():
            p.requires_grad = False
        model.fc = nn.Sequential(
            nn.Dropout(0.3),
            nn.Linear(model.fc.in_features, num_classes),
        )
    elif arch == "efficientnet_b0":
        model = models.efficientnet_b0(weights=models.EfficientNet_B0_Weights.I
        for p in model.parameters():
            p.requires_grad = False
        model.classifier[1] = nn.Linear(model.classifier[1].in_features, num_cl
    return model
```

4.3. Loop de treino com early stopping

python

```python
def train(model, train_dl, val_dl, epochs, lr, device):
    model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.AdamW(
        filter(lambda p: p.requires_grad, model.parameters()), lr=lr)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode="min", factor=0.5, patience=2)

    best_val_loss, patience, bad_epochs = float("inf"), 5, 0

    for epoch in range(epochs):
        model.train()
        train_loss = 0
        for x, y in train_dl:
            x, y = x.to(device), y.to(device)
            optimizer.zero_grad()
            loss = criterion(model(x), y)
            loss.backward()
            optimizer.step()
            train_loss += loss.item() * x.size(0)
        train_loss /= len(train_dl.dataset)

        model.eval()
        val_loss, correct = 0, 0
        with torch.no_grad():
            for x, y in val_dl:
                x, y = x.to(device), y.to(device)
                out = model(x)
                val_loss += criterion(out, y).item() * x.size(0)
                correct += (out.argmax(1) == y).sum().item()
        val_loss /= len(val_dl.dataset)
        val_acc = correct / len(val_dl.dataset)
        scheduler.step(val_loss)

        print(f"epoch {epoch+1} | train_loss {train_loss:.4f} | "
              f"val_loss {val_loss:.4f} | val_acc {val_acc:.4f}")

        if val_loss < best_val_loss:
            best_val_loss, bad_epochs = val_loss, 0
            torch.save(model.state_dict(), "models/best.pt")
        else:
            bad_epochs += 1
            if bad_epochs >= patience:
                print("Early stopping.")
```

```
                break
    return model
```

4.4. Execute as duas fases

```python
device = "cuda" if torch.cuda.is_available() else "cpu"
train_dl, val_dl = get_loaders()

model = build_model("resnet50")

# Fase 1: só a cabeça
train(model, train_dl, val_dl, epochs=8, lr=1e-3, device=device)

# Fase 2: descongela as últimas camadas (layer4 na ResNet)
model.load_state_dict(torch.load("models/best.pt"))
for name, p in model.named_parameters():
    if name.startswith("layer4") or name.startswith("fc"):
        p.requires_grad = True
train(model, train_dl, val_dl, epochs=15, lr=1e-5, device=device)
```

Registre tudo: salve as curvas de loss/acurácia por época (matplotlib) em `outputs/`. Compare ResNet50 × EfficientNet-B0 e documente qual venceu e por quê — isso é ouro para o README e para entrevistas.

Expectativa realista: com este dataset, algo entre 88% e 95% de acurácia de validação é atingível. Se estiver muito abaixo, revise normalização, learning rate e se o pré-processamento não está destruindo informação.

Etapa 5 — Avaliação: matriz de confusão e relatório de métricas

Acurácia sozinha não basta em sinistro: o custo de um **falso negativo** (carro danificado classificado como íntegro) é diferente do custo de um **falso positivo**. Analise isso explicitamente.

```python
```

```python
# src/evaluate.py
import numpy as np
import torch
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_auc_score, roc_curve)

@torch.no_grad()
def get_predictions(model, loader, device):
    model.eval().to(device)
    ys, preds, probs = [], [], []
    for x, y in loader:
        out = torch.softmax(model(x.to(device)), dim=1)
        probs.extend(out[:, 0].cpu().numpy())      # prob. da classe "dano"
        preds.extend(out.argmax(1).cpu().numpy())
        ys.extend(y.numpy())
    return np.array(ys), np.array(preds), np.array(probs)

def evaluate(model, val_dl, device, class_names=("damage", "whole")):
    y_true, y_pred, y_prob = get_predictions(model, val_dl, device)

    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=class_names, yticklabels=class_names)
    plt.xlabel("Predito"); plt.ylabel("Real")
    plt.title("Matriz de Confusão")
    plt.savefig("outputs/confusion_matrix.png", dpi=150, bbox_inches="tight")

    print(classification_report(y_true, y_pred, target_names=class_names))
    auc = roc_auc_score(1 - y_true, y_prob)   # AUC para detectar dano
    print(f"ROC-AUC (dano): {auc:.4f}")
```

Checklist da análise (escreva no notebook/README):

- Precision, recall e F1 **por classe** — qual classe o modelo erra mais?
- Quantos falsos negativos existem e o que significam no negócio (sinistro pago sem vistoria? fraude não detectada?).
- Inspecione visualmente 10–15 imagens mal classificadas: há padrão (reflexos, carros sujos, danos muito pequenos, fotos de interior)? Isso vira insight de melhoria.
- Se recall de dano for prioritário, discuta ajuste do threshold de decisão (ex.: classificar como dano se prob > 0.35 em vez de 0.5) e mostre o trade-off na curva ROC.

Etapa 6 — Grad-CAM: explicabilidade e localização visual do dano

Grad-CAM usa os gradientes da classe predita em relação à última camada convolucional para gerar um **mapa de calor** sobre a imagem, mostrando as regiões que mais influenciaram a decisão. É a sua "localização" do dano e o argumento de explicabilidade do projeto.

python

```python
# src/gradcam.py
import cv2
import numpy as np
import torch
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image
from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget

from src.preprocessing import preprocess
from src.dataset import val_tfms

def generate_gradcam(model, image_path, device, target_class=None):
    model.eval().to(device)
    target_layers = [model.layer4[-1]]           # ResNet50
    # EfficientNet: target_layers = [model.features[-1]]

    raw = preprocess(image_path)                  # RGB, 224x224, uint8
    rgb_float = raw.astype(np.float32) / 255.0
    input_tensor = val_tfms(image=raw)["image"].unsqueeze(0).to(device)

    with torch.no_grad():
        pred = model(input_tensor).argmax(1).item()
    target = target_class if target_class is not None else pred

    cam = GradCAM(model=model, target_layers=target_layers)
    grayscale_cam = cam(input_tensor=input_tensor,
                        targets=[ClassifierOutputTarget(target)])[0]

    overlay = show_cam_on_image(rgb_float, grayscale_cam, use_rgb=True)
    return overlay, pred
```

O que fazer com isso

1. Gere Grad-CAMs para **acertos das duas classes** e para **erros**. Salve em `outputs/gradcam/`.
2. **Avalie criticamente:** nos acertos de dano, o heatmap está sobre o amassado/arranhão ou sobre o fundo? Se o modelo acerta "olhando" para o lugar errado, você descobriu

um atalho espúrio (shortcut learning) — documente isso, é uma das análises mais valiosas do projeto.

3. Monte uma figura-resumo (original | heatmap | predição + confiança) para o README.

Etapa 7 — Publicação no Hugging Face Hub

7.1. Prepare a conta e o token

1. Crie conta em huggingface.co → Settings → Access Tokens → token com permissão **write**.
2. Autentique-se localmente:

```bash
huggingface-cli login
```

7.2. Faça o upload do modelo

```python
import torch
from huggingface_hub import HfApi, create_repo

repo_id = "SEU_USUARIO/car-damage-classifier"
create_repo(repo_id, exist_ok=True)

# salve o state_dict e um arquivo de config simples
torch.save(model.state_dict(), "models/best.pt")

api = HfApi()
api.upload_file(path_or_fileobj="models/best.pt",
                path_in_repo="best.pt", repo_id=repo_id)
api.upload_file(path_or_fileobj="README_model.md",
                path_in_repo="README.md", repo_id=repo_id)
```

7.3. Escreva a Model Card (README do repositório HF)

Inclua obrigatoriamente:

- **Descrição:** tarefa, arquitetura (ResNet50 fine-tuned), dataset e link do Kaggle.
- **Métricas:** acurácia, F1 por classe, AUC + imagem da matriz de confusão.
- **Exemplos de Grad-CAM** (imagens no repo).
- **Como usar:** snippet de código carregando o modelo com `hf_hub_download` + `load_state_dict`.
- **Limitações:** dataset pequeno, apenas binário, fotos de contexto específico; não usar como decisão final de sinistro sem revisão humana.

python

```python
# snippet de uso para a model card
from huggingface_hub import hf_hub_download
import torch
from src.train import build_model

path = hf_hub_download("SEU_USUARIO/car-damage-classifier", "best.pt")
model = build_model("resnet50")
model.load_state_dict(torch.load(path, map_location="cpu"))
model.eval()
```

Etapa 8 — API de inferência com FastAPI

8.1. Implemente o serviço

python

```python
# app/main.py
import io
import numpy as np
import cv2
import torch
from fastapi import FastAPI, UploadFile, File, HTTPException
from src.train import build_model
from src.dataset import val_tfms
from src.preprocessing import resize_with_padding, denoise, apply_clahe

app = FastAPI(title="Car Damage Classifier API")
CLASSES = ["damage", "whole"]

device = "cuda" if torch.cuda.is_available() else "cpu"
model = build_model("resnet50")
model.load_state_dict(torch.load("models/best.pt", map_location=device))
model.eval().to(device)

def preprocess_bytes(data: bytes) -> np.ndarray:
    arr = np.frombuffer(data, np.uint8)
    img = cv2.imdecode(arr, cv2.IMREAD_COLOR)
    if img is None:
        raise ValueError("Arquivo não é uma imagem válida.")
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    return apply_clahe(denoise(resize_with_padding(img)))

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    if file.content_type not in {"image/jpeg", "image/png"}:
        raise HTTPException(415, "Envie JPEG ou PNG.")
    try:
        img = preprocess_bytes(await file.read())
    except ValueError as e:
        raise HTTPException(422, str(e))

    tensor = val_tfms(image=img)["image"].unsqueeze(0).to(device)
    with torch.no_grad():
        probs = torch.softmax(model(tensor), dim=1)[0]
    pred = int(probs.argmax())
    return {
        "prediction": CLASSES[pred],
        "confidence": round(float(probs[pred]), 4),
```

```
        "probabilities": {c: round(float(p), 4) for c, p in zip(CLASSES, probs)}
    }
```

8.2. Rode e teste

```bash
uvicorn app.main:app --reload
```

- Acesse `http://localhost:8000/docs` — o FastAPI gera a documentação Swagger automaticamente, com upload interativo.
- Teste via curl:

```bash
curl -X POST http://localhost:8000/predict \
  -F "file=@exemplo_dano.jpg"
```

8.3. Melhorias que valem pontos extras

- Endpoint `/predict-with-gradcam` que retorna a imagem com heatmap em base64.
- Carregar o peso direto do Hugging Face Hub no startup (`hf_hub_download`), em vez de arquivo local.
- `Dockerfile` para containerizar (python-slim + uvicorn) — facilita deploy em Render/Railway/HF Spaces.

Etapa 9 — Documentação final (README do projeto)

O README é o que transforma código em portfólio. Estruture assim:

1. **Problema de negócio:** triagem automática de fotos de sinistro — por que isso economiza tempo/dinheiro para uma seguradora.
2. **Pipeline (diagrama):** foto → OpenCV → CNN → predição + Grad-CAM → API.
3. **Dataset e EDA:** números e observações da Etapa 1.
4. **Metodologia:** pré-processamento, augmentation, transfer learning em 2 fases, arquiteturas comparadas.
5. **Resultados:** tabela de métricas, matriz de confusão, curva ROC, exemplos de Grad-CAM (acertos e erros comentados).
6. **Limitações e próximos passos.**
7. **Como rodar:** setup, treino, avaliação e API em comandos copiáveis.
8. **Links:** modelo no HF Hub, dataset no Kaggle.

Extensão opcional — Segmentação de imagem (para marcar o checkbox de segmentação)

O dataset do Kaggle não traz máscaras, então para segmentação real você tem dois caminhos:

1. **Anotar um subconjunto:** use uma ferramenta como CVAT ou Label Studio para desenhar máscaras de dano em ~150–300 imagens e treine um **U-Net** (com encoder pré-treinado via `segmentation-models-pytorch`) ou faça fine-tuning de um **Mask R-CNN** do torchvision. É trabalhoso, mas demonstra domínio completo do ciclo (anotação → treino → avaliação com IoU/Dice).
2. **Usar um dataset já anotado:** o dataset **CarDD (Car Damage Detection)** possui máscaras de segmentação de danos por categoria (amassado, arranhão, trinca, etc.) e é a rota mais rápida para um modelo de segmentação de verdade.

Em ambos os casos, as métricas passam a ser **IoU (Jaccard)** e **Dice**, e o Grad-CAM ganha um "gabarito" interessante: você pode comparar o heatmap da classificação com a máscara real do dano.

Checklist de conclusão

- [ ] Ambiente reprodutível (`requirements.txt` + README de setup)
- [ ] EDA documentada com contagens e observações
- [ ] Pipeline OpenCV validado visualmente e reutilizado em treino/avaliação/API
- [ ] Augmentation justificado e conferido em batches reais
- [ ] Transfer learning em 2 fases, com curvas de treino salvas
- [ ] Comparação de ao menos 2 arquiteturas (ResNet50 × EfficientNet-B0)
- [ ] Matriz de confusão + classification report + ROC-AUC + análise de erros
- [ ] Grad-CAM em acertos e erros, com análise crítica
- [ ] Modelo publicado no HF Hub com model card completa
- [ ] API FastAPI funcional com `/health` e `/predict` testados
- [ ] README de portfólio com resultados e limitações